How to Work in the Power Cloud OS

The Power Cloud workspace guide · Version 2.0 draft · August 12, 2026

> *Updated 2026-08-22: the deck deliverable is now the **Presentation** (was Proposal), and function 11 covers feedback on the OS.*

This workspace is where Power Cloud client work happens: assessments, implementations, execution and build work, support tasks, status reporting, and data work. Claude Code does the heavy lifting; your job is to point it at the right client, review at checkpoints, and own everything that reaches a customer.

One idea explains the whole system: **the folder is the memory.** Everything about a client lives in their folder, so any consultant's session picks up with full context. "Anybody continuing a conversation in that same folder will be talking to a being that understands it."

This guide assumes Windows 11. Already set up? Skip to Part 4.

Part 1: One-time software setup

Do this once per machine. Budget 30 to 45 minutes. If someone is onboarding you, do this part together; a second set of eyes catches the problems you can't see from your side, like a seat that was never granted.

1.1 Google Drive for desktop

1. Download from **https://www.google.com/drive/download/**, install with defaults.
2. Sign in with your **@powerclouderp.com** account.
3. Confirm `G:\Shared drives\Power Cloud OS` opens in File Explorer. If **Shared drives** is missing, ask your admin for access.

*Drive streams files on demand. If a file looks empty, right-click → **"Available offline."***

1.2 Git

From **https://git-scm.com/download/win**, install with all defaults. Verify: `git --version` in PowerShell.

1.3 Python

From **https://www.python.org/downloads/**: check **"Add python.exe to PATH"** on the first screen, then Install. Verify with `python --version`, then:

```
python -m pip install python-docx python-pptx comtypes pillow
```

1.4 Brand fonts

Download **Montserrat** and **Barlow** from Google Fonts (or use the copies in this workspace's `fonts/` folder). Select all `.ttf` files → right-click → **"Install for all users"** (per-user installs break PDF export). Microsoft Office must also be installed.

1.5 Claude Code

Desktop app or CLI, signed in with your Power Cloud account. Open the workspace once and accept the plugin trust prompt. Confirm the team's NetSuite connectors appear in the session's server list (authorize the ones you need via claude.ai connector settings).

1.6 Transcription key (set it up the first time you need it)

When a client hands over a call recording with no transcript, Claude transcribes it through AssemblyAI. That takes a key of your own, kept on your machine. Sign up at **https://www.assemblyai.com/** (free credit on signup, no card), copy the API key from the dashboard, then in PowerShell:

```
[Environment]::SetEnvironmentVariable('ASSEMBLYAI_API_KEY', '<your key>', 'User')
```

Open a new terminal afterwards. The key never goes in the workspace or a client folder. Client recordings are transcribed only after the client's consent is logged in their CONTEXT.md; the consent pack's Block 1 carries the disclosure.

Part 2: Engagement prerequisites (per client, do these first)

Several involve other people and have lead time, so they all go out at engagement start in one email: the **access & consent pack** (`templates/engagement/access_and_consent_pack.md`). The scaffolder drafts it for you (Part 5, function 1). The pieces:

2.1 Tell the customer we use AI and will connect via MCP

Before connecting anything, the client gets the disclosure: we connect to their NetSuite through MCP (read-only during discovery) and use AI in our process. This is a conversation and a written yes, not a formality. The approval gets logged in the client's CONTEXT.md with the date.

2.2 Obtain NetSuite access (production and sandbox, both)

Ask for **both up front**. Production gives the freshest picture for assessment work (sandbox data is frozen at its last refresh, which has burned findings before); sandbox is where execution builds and tests later. Assessment work is read-only either way, so production access during discovery carries no write risk. If only one comes through at first, take production for discovery and keep chasing the sandbox; it has lead time and you'll need it the moment work is approved.

2.3 Script Reader install (Mike, Taha, or Dev only)

The read-only SDF toolset that lets findings describe what a script *does* rather than that it exists. Needs the client's go-ahead; the consent pack carries the notice. Without it, script claims stay [Reported], never [Verified].

2.4 MCP connection added centrally

Send **Mike or Taha the customer's account ID**; they add the connection for the whole team. Connections carry the environment in their name ("- Prod", "- SB"). That naming matters: the write guardrail reads it.

2.5 Other systems in their stack

Shopify or QuickBooks in the stack? The consent pack has blocks for those connections too; see `DATA_CONNECTORS.md` for how each works. Credentials never land in this Drive; they live on the connection holder's machine.

> ***Checklist before real work:*** *disclosure approved → sandbox access → Script Reader installed → MCP connection live → identity check passed (the account actually belongs to this client).*

Part 3: The folder structure (read once)

Every client lives in its own subfolder here. Inside each:

```
{ClientName}/
├── CONTEXT.md       living fact sheet: status, contacts, access log, scope
├── source/          client-provided material (READ-ONLY)
├── working/         scratch: evidence, drafts, profiles, anything in progress
├── deliverables/    finals only
└── execution/       created when build work starts; execution artifacts live here
```

Rules that matter from day one:

- **Never edit `source/`.** Client words and data. Transform via copies in `working/`.
- **Never open another client's folder.** Siblings are confidential. Cross-client learnings live only in the shared root files.
- **Legacy history stays where it is.** Long-standing clients have a folder under `G:\Shared drives\Power Cloud Project Folders\`; that's a read-only annex, pointed to from CONTEXT.md, never reorganized. Copy in only what an engagement actively needs, properly exported (never Explorer-copy a `.gdoc`).
- **The Monday hours print lands in `working/`.** Every Monday at 9:30 ET, each client's NetSuite time entries are written to their own `working/time_entries_{date}.json`, eight weeks back. That only happens if the folder exists and its CONTEXT.md carries the NetSuite job name, which is what setup captures (Part 5, function 1).
- **Meeting recordings arrive through Paco.** Paco is Power Cloud's Google Chat assistant. When a Meet call you organized produces a recording, a transcript, or Gemini notes, Paco sends you a private Chat card within about 15 minutes with the meeting and a client button (or "Choose a client" if it wasn't sure). Click the client, or reply with the client's name in that thread, and Paco moves the originals into the client's `source/`, writes a readable `.md` copy of the notes beside them, logs it in `working/meetings_log.md`, and tells you when it's done. That's the whole job: no second confirmation, no filing by hand. A meeting with nothing recorded produces no card at all, which is normal. Putting the client's full name in your meeting titles is what gets you the right button first time. Everything Paco writes is generated: read it, never edit it.
- **A Monday board update lands in your project chat.** Every Monday at 10:00 ET, for each client whose CONTEXT.md names both an Asana project and a project chat space, Paco posts two messages into that space: a link to the week's executive summary if one has been produced, and the Asana tasks that need attention (past due, due this week, active work untouched for a fortnight, and anything with no owner or no due date). It is an internal view and says so; Paco reads the space membership first and refuses to post if anyone outside Power Cloud is in there. Nothing in it is written by hand, so the way to change what it says is to fix the board. Paco does not read replies in the space.

You'll also see `.md` files at the workspace root (playbooks, writing standards, capability registers). Those are Claude's operating instructions; it reads them automatically and stays inside their rules on its own. You don't need to read or maintain them. Two are for people rather than for Claude: `FEEDBACK.md`, the running

list of what is wrong with the tool (see function 11), and `ENGAGEMENT_LOG.md`, which shows what is in flight across the practice. Both are generated, so read them and never edit them.

Part 4: Start a session

Every session starts inside the client's folder under `G:\Shared drives\Power Cloud OS`. The only question is whether the folder exists yet.

New client, or first new work for an old client:

1. In File Explorer, create a folder named for the client under `G:\Shared drives\Power Cloud OS`. Plain client name only ("Teppermans", "Acme Apparel"), no "RA" or engagement suffixes; the folder outlives any one engagement.
2. Drop whatever documents you have into it, loose is fine: sales notes, transcripts, exports.
3. Open **Claude** → **Code** in that folder and say:

   "New client. Run session start."

Claude files your loose documents into `source/`, builds the rest of the structure, and interviews you for the fact sheet: engagement type, contacts, tech stack, billing channel. For an old client it asks about their historical folder under `G:\Shared drives\Power Cloud Project Folders` and links it as read-only history, so you don't copy old documents over; it reads them where they live and pulls in only what the engagement actually needs. It also checks for an existing Asana project before creating anything, and drafts the access & consent pack for your review.

Customer we support but never set up: somebody asks you for a status report, or for hours, on a customer with no folder here. That request is the trigger, not a reason to say no. Create the plain-name folder, open **Claude** → **Code** in it, and say:

"Set up Absci. We support them but they were never onboarded."

Same interview, shorter. It skips the sales questions and asks only what the rest of the OS needs to work. A customer earns a folder when we produce work product for them; being billed once for a one-off doesn't.

Existing client: open their folder (`G:\Shared drives\Power Cloud OS\{ClientName}`; quote the path in a terminal, it has spaces) and say:

"This is the {ClientName} engagement. Run session start."

It reads the client's fact sheet, inventories the folder, confirms the connected NetSuite account actually belongs to this client, and checks which tools are available. Then tell it what you want; Part 5 is the menu. Same starting point for every phase of work, discovery through execution.

Either way, session start ends by suggesting a session name as a paste-ready line (like `/rename Teppermans SB - suitelet fix`). Paste it and the session is labeled; ignore it and the auto-generated title still roughly works.

Session best practices

These are the habits that separate people who get a lot out of this tool from people who fight it.

- **One task, one session.** Start a clean session for each new task or client. Your files always survive: everything written to the client folder persists across sessions and clears. Clearing a session erases conversation memory, never work. The folder is the memory.

- **Name your sessions.** Sessions title themselves from your first message, which is fine for one session at a time. When you're juggling several, paste the `/rename` line Claude suggests at session start ("Teppermans SB - suitelet fix"), or rename from the desktop app's session list. Generic titles have already caused production/sandbox confusion.
- **Run several sessions at once.** Different clients, different tasks, one task each. While one works, review another's output or feed a third. Named sessions are what make this manageable, and it's the difference between waiting on the tool and managing it.
- **Work account only.** Check you're signed into your Power Cloud account, not a personal one, before starting real work. The wrong account loses the team's context and connections.
- **Brainstorm and plan together before you delegate.** The best sessions start as a conversation, not an order: talk through the problem, ask what it would do, challenge its approach, let it interview you. The plan is where your judgment matters most, so spend your time there.
- **Then give it a chunk and walk away.** Once the plan is solid, hand over the whole thing: "I'm away for two hours; work the plan, and where you'd normally ask me, use your best judgment and log the decision." Plan together, execute apart. Small piecemeal requests waste your time and its capability.
- **Never watch it type.** Waiting is not billable. Line up parallel work: another client, a draft to review, the thing you postponed.
- **You're managing, not typing.** Check in like you would on a colleague you delegated to: read what it did, correct course, hand it the next chunk.
- **Correct it out loud.** When it gets something wrong and you fix it, say so in the session and tell it to remember. That's how a correction becomes a permanent rule instead of next week's repeat.
- **Picking up where you left off works.** Reopen the client folder and say "pick up where you left off." Prefer that over re-explaining from scratch.
- **Read everything before you trust it.** It produces polished-looking output at speed; that's exactly what makes skipping review tempting and dangerous. Proofread every draft like a college kid's midterm essay before it informs a decision or leaves the building.
- **When a guardrail prompt appears, read it.** A blocked write or a confirmation request is the system protecting a client account, not an error. If a production-write confirmation shows up during discovery work, the answer is decline.

Part 5: The functions, with sample prompts

Each function: when to use it, what to say, what happens, and where you stay in the loop. Prompts are starting points. Say it your way; the skills carry the process.

1. Set up a client (new, or first work on an old one)

When: the client has no folder here yet, or has one that was never filled in. This runs inside the folder you just created (Part 4).

> *"New client. Run session start."*
>
> *"This is new work for Lulus, an old client. Set them up."*
>
> *"Foundrae is an in-flight project we're bringing into the workspace. Set them up."*
>
> *"Set up Absci. We support them but they were never onboarded."*

It files whatever you dropped in, checks for an existing Asana project and folder-name variants *before* creating anything, interviews you (engagement type, website, billing channel, stack, contacts; for in-flight work: phase, current agreement, existing access), pre-fills CONTEXT.md, locates the legacy folder and

links it as read-only history, and drafts the access & consent pack for you to review and send. Adopting an in-flight project should take an hour, not a day.

The answer that matters most is the NetSuite job name, exactly as it reads in Power Cloud's own NetSuite (`44 Pro Gloves`, `N. Teppermans Limited`, `Goodheart-Wilcox SCA`). It is a descriptive job title rather than a tidy client name, and it often doesn't match the folder name at all. That one line is what routes the Monday hours print into the folder, which is what the executive summary runs on. If a client bills to two jobs, say so and both get recorded. If you don't know the exact string, say that instead of guessing, and it becomes an open question that keeps surfacing. Two clients were found dropping 181 and 243 hours respectively because that line was never filled in, and nothing visibly failed in either case.

Already have a folder that's missing things? Run the same prompt. It reads what's there, asks only about the gaps, and never overwrites an answer that's already good.

2. Prep for a kickoff or onsite

When: after the sales handoff, before you're in front of the client.

> *"We got the handoff from sales. Prep me for the kickoff call, and draft the questions we should send the client in advance."*
>
> *"Prep for the onsite."*

It mines the sales notes and transcripts, runs read-only MCP discovery against the account, researches the client's public website, and produces a question list and time-boxed agenda. The client-facing one-pager waits for your review (checkpoint 1). The effect on the call: "before the kickoff, they're asking you questions which nobody even asked."

3. Run the assessment (Rapid Assessment or Fast Track Discovery)

When: discovery sessions are done, transcripts are in `source/`.

> *"The interviews are complete. Produce the handover docs."*

The heavy machine: exhaustive evidence mining with a candidate register (nothing discussed can silently vanish), MCP verification with evidence tiers ([Verified]/[Reported]/[Inferred]), solution research against Oracle docs, and the three deliverables: assessment document, presentation deck, execution plan. You review at findings (checkpoint 2), drafts (checkpoint 3), and **line-by-line on every LOE figure** (checkpoint 4). Claude drafts the numbers; you own them.

Only a recording, no transcript? Say *"Transcribe the recording in source first"*, or just start the assessment and the skill spots the gap. It checks that the client's consent is logged and your key is set (Part 1.6), produces a speaker-labeled transcript in `working/transcripts/`, and asks you to confirm who Speaker A, B and C are before any name lands on a quote.

4. Execute approved work

When: the client signed off; it's time to build.

> *"Let's start execution. Work on tasks O2C.3 and W.1."*
>
> *"Do Phase 1."*

Runs write-enabled under the execution guardrails: research first, plan for your approval (gate 1), build, deploy to **sandbox**, test against acceptance criteria, then promote to production behind an explicit approval per deploy (gate 2). Production writes force a confirmation prompt mechanically; if you ever see that prompt during discovery, decline it.

5. Work a support task or ticket

When: ongoing clients, one-off fixes, the daily queue.

> *"Pull the Asana task 'Price tag reprint bug' and plan the fix."*
>
> *"The client says component tags print when they shouldn't. Dig into the script with ScriptReader and tell me why."*

Same discipline in miniature: investigate (read the actual script source, query the account read-only), plan, get your approval, build sandbox-first. The Tepperman's suitelet went from transcript to sandbox-deployed fix in a day this way: "nobody touched the script, only Claude touched the script."

6. Generate the executive summary

When: before every status call, on the client's cadence.

> *"Run the weekly executive summary for Tepperman's."*

Trailing two full weeks of Asana activity, meetings, and hours become a client-ready summary plus deck. **Always reviewed before it goes out; permanent policy.** It also shows you your own data quality: when the summary says "no next steps recorded," that's the Asana hygiene talking (see `ASANA_CONVENTIONS.md`).

If the customer has no folder here, set them up first (function 1, a few minutes), then come back. Without a folder there is nowhere for the hours to land and nowhere to write the draft. Hours arrive with Monday's run, so a customer set up on a Thursday has no hours until the following Monday. Say that to whoever asked rather than reporting a number that only looks complete.

The summary you run is the one Paco posts. If the client has a project chat space, Monday's board update links whatever internal draft is sitting in `working/` dated within the last seven days. So a summary run on Friday is what the team sees on Monday, and a week with no run simply says so. Running it does not send anything to the client; that still takes a human.

7. Draft client communications

When: status updates, proceed-gates, billable-rework notices: anything where wording carries commercial weight.

> *"Draft the proceed-gate message: discovery hours are consumed, low-hanging fruit is done, we want the go-ahead for the fixes at 12 hours."*
>
> *"Draft a billable-rework notice for the reprint change. Factual, not apologetic."*

You send; it drafts. Everything client-facing follows `WRITING_STANDARDS.md`: brass tacks, no novels, no apology tone.

8. Do data work

When: the client's stack includes Shopify/QuickBooks, or a migration is on the table.

> *"They sent their item master export. Profile it and flag the design decisions before we map anything."*
>
> *"Connect read-only to their QuickBooks per DATA_CONNECTORS.md and produce the data profile."*

The pipeline: profile → design decisions surfaced explicitly (matrix vs attributes, customer hierarchy, one-way-door settings) → mapping spec you approve → sandbox dry run → reconciliation report → gated load. The mapping spec is the contract; you review it, not ten thousand rows.

9. Ask anything, with the client's full context

When: constantly. This is what the folder-as-memory exists for.

> *"Based on everything in this folder, what should I know before tomorrow's call with their CFO?"*
>
> *"In your opinion, are we missing any low-hanging fruit on this account?"*
>
> *"The client asked if NetSuite can do X natively. Check Oracle's docs and give me a straight answer."*

The folder is the memory: every prior call, finding, and decision informs the answer. One habit makes these answers trustworthy: when a claim smells off, ask "what's your source?" and make it trace the answer back.

10. Close out and learn

When: finals shipped, phase done, or end of day.

> *"Finals shipped. Run the retro."*
>
> *"Wrap up: update CONTEXT.md with today's decisions and what's open for next session."*

Retros harvest corrections into the shared baselines (LOE actuals, playbook patterns, capability learnings), with you deciding what gets promoted. This loop is why the process gets better every engagement.

11. Tell us the tool got in your way

When: the moment it happens. That is the whole trick, because the annoyance is forgotten by the end of the day.

> *"Log this as feedback."*
>
> *"/feedback the deck render fails every time on a client logo"*
>
> *"This should have caught that before I spent an hour on it. File it."*

That covers anything about the OS itself: something that broke, something that wasted your time, a rule that is wrong, or a feature that should exist. Claude writes the entry from what already happened in the session, so you are not filling in a form. It asks only the three things it cannot work out on its own: **how much time it cost you, whether you were blocked, and whether it happens every time.** If it cost real time, it will ask two more and offer a couple of concrete fixes to react to. "Not sure, you figure it out" is a perfectly good answer.

It will also offer, once, when it notices friction it caused. Say no and it drops it.

Entries land in `feedback/` at the root, ranked by cost and frequency in `FEEDBACK.md`, and get worked through periodically. Everything leaves the list either as a change to the system or as a decline with a reason. Nothing is quietly dropped.

This is not the same as the retro. The retro captures what we learned about NetSuite and about running an engagement. This captures what is wrong with the tooling. Client account matters belong in neither; they stay in that client's own files.

Part 6: The rules that never bend

1. **Read-only during discovery.** No writes to any client system during assessment or discovery. No exceptions. Writes happen in execution, sandbox-first, human-gated.
2. **No production write ever happens silently.** Every prod write forces an explicit confirmation. During discovery, the answer to that prompt is decline.
3. **Credentials never touch this Drive.** Tokens and secrets live on the connection holder's machine. If you find credentials in a folder, flag them.

4. **Everything client-facing gets human eyes before it ships.** You own what you send. "The AI wrote it" is the failure, not the defense.
5. **LOE numbers are commercially binding once sent.** Claude drafts them, a human owns them, line by line.
6. **Consent is a record.** AI/MCP disclosure, installs, and write authorizations get logged in CONTEXT.md with dates.
7. **Check before create.** Folders, Asana projects, email groups: look for the existing one first.

Part 7: The checkpoints, in one place

#	Checkpoint	When
1	Pre-onsite agenda review	Before the one-pager goes to the client
2	Findings review	After evidence + verification, before drafting
3	Draft review	After markdown drafts, before rendering finals
4	**LOE & commercial review**	Every figure and the billing terms, line by line
5	Plan approval (execution gate 1)	Before any build starts
6	Production deploy (execution gate 2)	One human approval per deploy

Part 8: Gotchas and quick fixes

- **Paths**: quote them; the workspace path has spaces.
- **File looks empty**: Drive hasn't synced. Right-click → "Available offline."
- **`python`/`git` not recognized**: PATH issue; new PowerShell window or reinstall with the PATH box checked.
- **Wrong fonts in a render**: fonts not installed for all users (Part 1.4).
- **`ASSEMBLYAI_API_KEY` not set**: the transcription key is missing on this machine (Part 1.6). Open a new PowerShell window after setting it.
- **Wrong NetSuite account**: Claude stops and asks. Don't override; fix the connection.
- **Sandbox looks quiet**: sandbox data is frozen at the last refresh. Not evidence of a production problem.
- **A write got blocked or forced a confirmation**: the guardrail working, not an error. Read the message.
- **Never put client data in a web search.** Public identity (their website, history) is fine and expected; account data, configs, and financials are not.
- **Claude says you've hit a usage limit**: tell Mike. Not a failure; it's how seat sizes get right-sized.
- **No board update appeared on Monday**: the client needs both an `Asana project` and a `Project chat` line in its CONTEXT.md, and Paco has to be a member of that space. Run `python "../.claude/scripts/check_board_mapping.py"` in the client folder to see which is missing. A space holding anyone outside Power Cloud is refused on purpose.
- **Paco didn't ask about my meeting**: it only prompts the organizer, only when there's a recording, transcript, or notes, and only for meetings after your rollout date. Check those three first. Still missing after an hour, or a card whose buttons fail: tell Mike.

- **Paco suggested the wrong client, or none**: pick the right one from the card (that files it correctly), then ask Claude to add the name Paco missed to the `Meeting aliases` line in that client's CONTEXT.md so it matches next time.
- **A `.gdoc` in `source/` won't open**: it's a Drive link to the original Google Doc that Paco moved. The readable copy is the `- notes [...].md` file beside it; Claude reads that one.

Quick reference: a day in the OS

- New client (or first work on an old one) → create a plain-name folder → drop your docs in → open it → *"New client. Run session start"*
- Existing client, any work → their folder → *"Run session start"*, then say what you need (functions 2-11)
- Asked for a report on a customer with no folder → create the folder → *"Set up {Client}. We support them but they were never onboarded."*
- Before a status call → *"Run the executive summary"* → review → send
- Before anything reaches the client → your eyes, your name, your call
- Anything about the tool annoyed you → *"Log this as feedback"* or `/feedback`, right then, not later